Experiment 10: Implement CI/CD Pipeline Using GitHub Actions and Render

Aim

To set up automated continuous integration and continuous deployment (CI/CD) pipeline using GitHub Actions to automatically test and deploy Flask applications to Render.

Software Requirements

- GitHub account and repository
- Render.com account
- Flask application (from previous experiments)
- GitHub Actions (built-in with GitHub)
- Git command line tool

Theory

CI/CD automation streamlines the development-to-production workflow. Continuous Integration (CI) runs tests automatically when code is pushed, catching errors early. Continuous Deployment (CD) automatically deploys successful builds to production. GitHub Actions is a workflow automation tool that triggers on GitHub events (push, pull request, etc.). Render is a cloud platform that hosts web applications, automatically detecting code changes and redeploying. This experiment demonstrates how changes pushed to GitHub automatically trigger tests and deploy to Render without manual intervention.

Procedure

Step 1: Create directory and virtual env

```
mkdir exp10_ci_cd_github cd
exp10_ci_cd_github python -m venv
venv
# Windows:
venv\Scripts\activate #
Mac/Linux:
source venv/bin/activate
```

Step 2: Install requirements

```
pip install flask flask-jwt-extended pymongo bcrypt python-dotenv
```

Step 3: Prepare Flask Application

> **Important**
>
> The code required is same as experiment 7. Either manually create or copy following files:
>
> ```
> cp ../exp7_jwt_auth/app.py .
> mkdir templates static static/css static/js
> cp ../exp7_jwt_auth/templates/index.html templates/ cp
> ../exp7_jwt_auth/static/css/jwt.css static/css/ cp
> ../exp7_jwt_auth/static/js/jwt.js static/js/
> ```

Ensure your Flask app has:

- `app.py` - Main application
- `requirements.txt` - All dependencies (Run `pip freeze > requirements.txt`)
- `Procfile` - Instructions for Render on how to start the app

Create `Procfile`:

```
web: gunicorn app:app
```

Step 4: Add Gunicorn to requirements.txt

Update your `requirements.txt`:

```
pip install gunicorn
pip freeze > requirements.txt
```

Your requirements.txt should include:

```
bcrypt==5.0.0
blinker==1.9.0
click==8.3.1
dnspython==2.8.0
Flask==3.1.2
Flask-JWT-Extended==4.7.1 gunicorn==23.0.0
...
```

Step 5: Create GitHub Repository

1. Go to https://github.com/new
2. Create repository named `fullstack-ai-lab`
3. Add description: "Full Stack AI Lab with Flask, MongoDB, ML"
4. Choose Public (for free GitHub Actions)
5. Click "Create repository"

Step 6: Create `.gitignore` file

Create a fine .gitignore with following content:

```
.env venv/
__pycache__/
```

Step 7: Initialize Git and Push Code

> **Tip**
>
> These commands are listed once the repository is created. Just make sure that you are running `git add .` instead of `git add README.md`.

In your project directory:

```
git init git add .
git commit -m "Initial commit: Flask app with authentication"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/fullstack-ai-lab.git git push -u origin main
```

Step 8: Create GitHub Actions Workflow

Create directory structure:

```
mkdir -p .github/workflows
```

Create `.github/workflows/ci-cd.yml`:

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Set up Python
      uses: actions/setup-python@v4
      with:
        python-version: '3.10'

    - name: Install dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt pip
        install pytest pytest-cov

    - name: Run tests
      run: |
        pytest --cov=. --cov-report=xml
```

```
    - name: Upload coverage reports
      uses: codecov/codecov-action@v3
      with:
        files: ./coverage.xml

  deploy:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'

    steps:
    - name: Checkout code
      uses: actions/checkout@v3

    - name: Deploy to Render
      env:
        RENDER_DEPLOY_HOOK: ${{ secrets.RENDER_DEPLOY_HOOK }}
      run: |
        curl $RENDER_DEPLOY_HOOK
```

Step 9: Create Basic Tests

Create `test_app.py`:

```
import pytest
from app import app

@pytest.fixture def
client():
    """Flask app test client"""
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

def test_home_route(client): """Test home page
    loads""" response = client.get('/')
    assert response.status_code == 200

def test_signup_missing_data(client): """Test signup with
    missing email"""
    response = client.post('/api/signup', json={'password': 'test123'}) assert response.status_code == 400
    assert 'error' in response.json

def test_protected_route_no_token(client): """Test protected
    route without token""" response =
    client.get('/api/protected') assert response.status_code ==
    401
```

Step 10: Set Up Render Deployment

1. Go to https://render.com and sign up (TIP: Login to GitHub to make things easier)
2. Click "New +" → "Web Service"
3. Connect your GitHub repository
4. Select `fullstack-ai-lab` repository
5. Configure settings:
    - Name: `fullstack-ai-lab-prod`
    - Environment: `Python 3.10`
    - Build Command: `pip install -r requirements.txt`
    - Start Command: `gunicorn app:app`
    - Add Environment Variables:
        - `MONGO_URI`: Your MongoDB connection string
        - `JWT_SECRET_KEY`: Your secret key
        - `SECRET_KEY`: Your secret key
6. Click "Create Web Service"
7. Go to Settings → Deploy Hook
8. Copy the Deploy Hook URL

Step 11: Add Deploy Hook as GitHub Secret

1. Go to your GitHub repository
2. Settings → Secrets and variables → Actions
3. Click "New repository secret"
4. Name: `RENDER_DEPLOY_HO`
5. Value: Paste the Render Deploy Hook URL
6. Click "Add secret"
7. Do the same for `MONGO_U` (Needed for tests)

Step 12: Push Workflow File to GitHub

```
git add .github/ git add
test_app.py
git commit -m "Add CI/CD pipeline and tests" git push origin
main
```

Step 13: Monitor Workflow Execution

1. Go to your GitHub repository
2. Click "Actions" tab
3. Watch the workflow execute:
    - Tests run on Ubuntu
    - If all tests pass, deployment to Render triggers
    - Monitor deployment status

Step 11: Verify Deployment

1. Go to Render dashboard
2. Click your service
3. Check "Events" log for deployment status
4. Once deployed, access your app at the Render URL

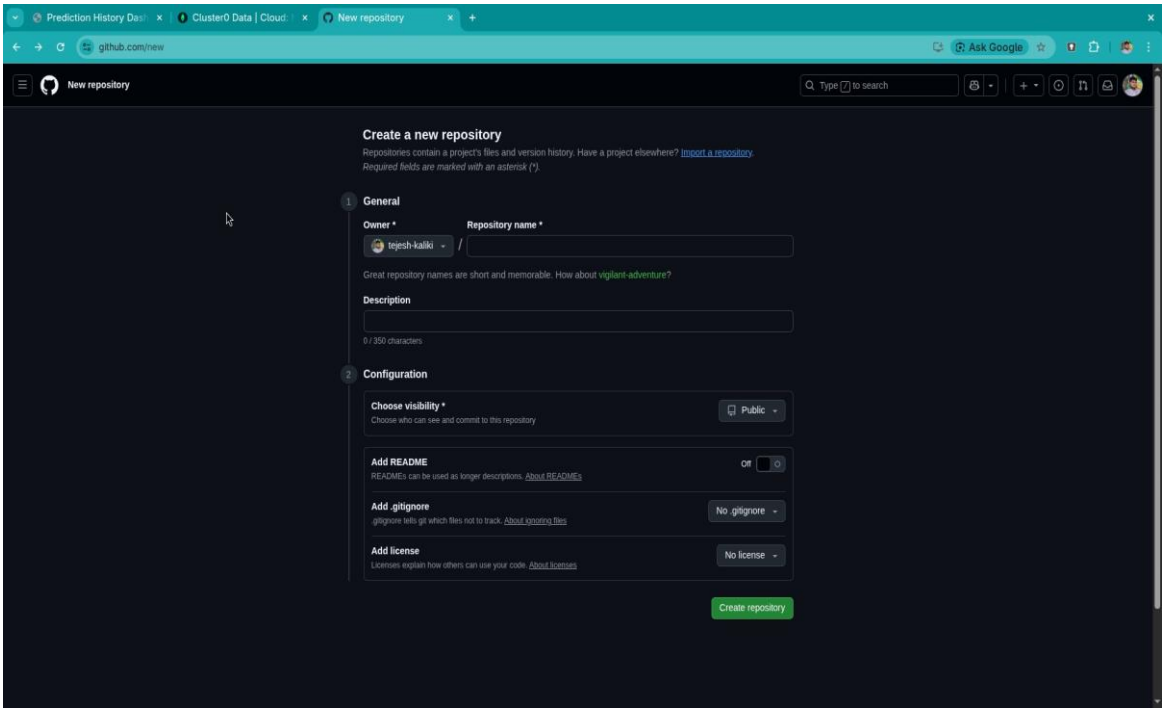
Output

A complete CI/CD pipeline with:

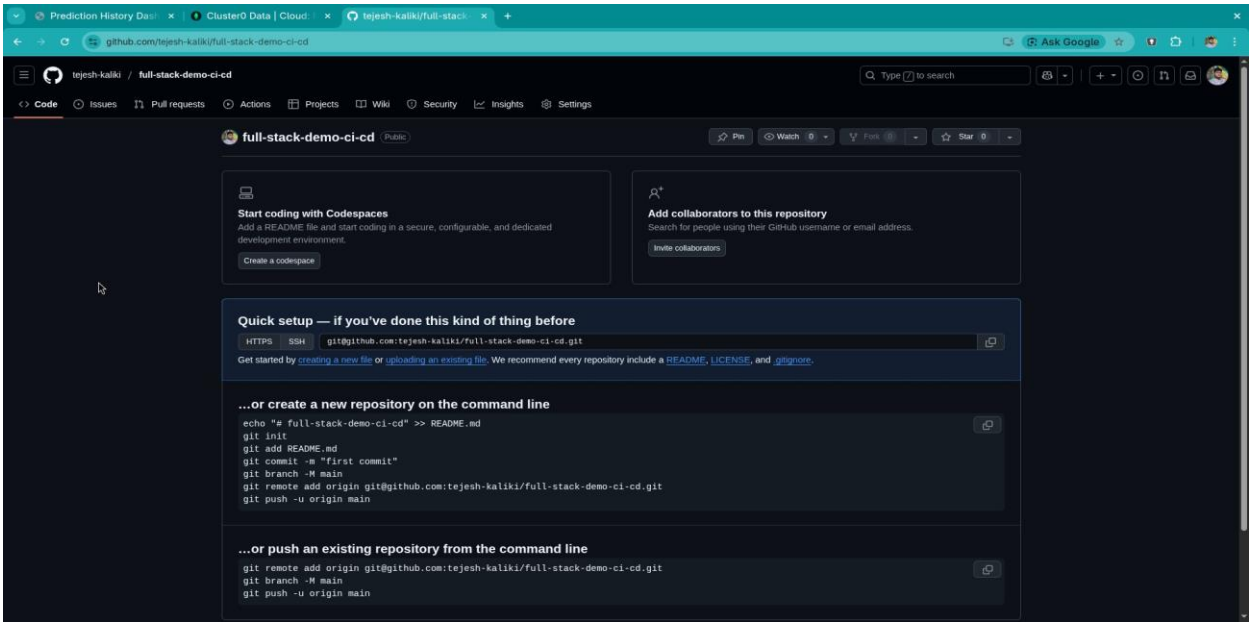
- Automatic tests on every push
- Code coverage reporting
- Automatic deployment to Render on successful tests
- Deploy hooks for one-click redeployment
- GitHub Actions workflow logs
- Environment variable management
- Production-ready deployment
- Zero-downtime deployments

Screen-shots:

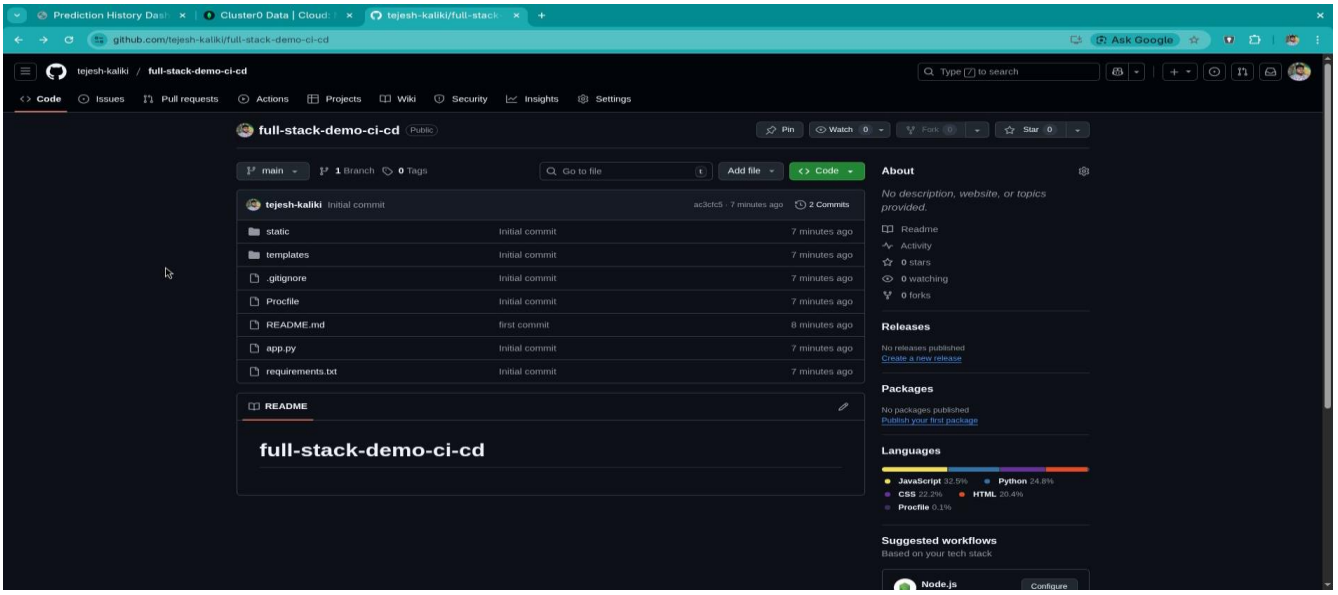
New repository create page



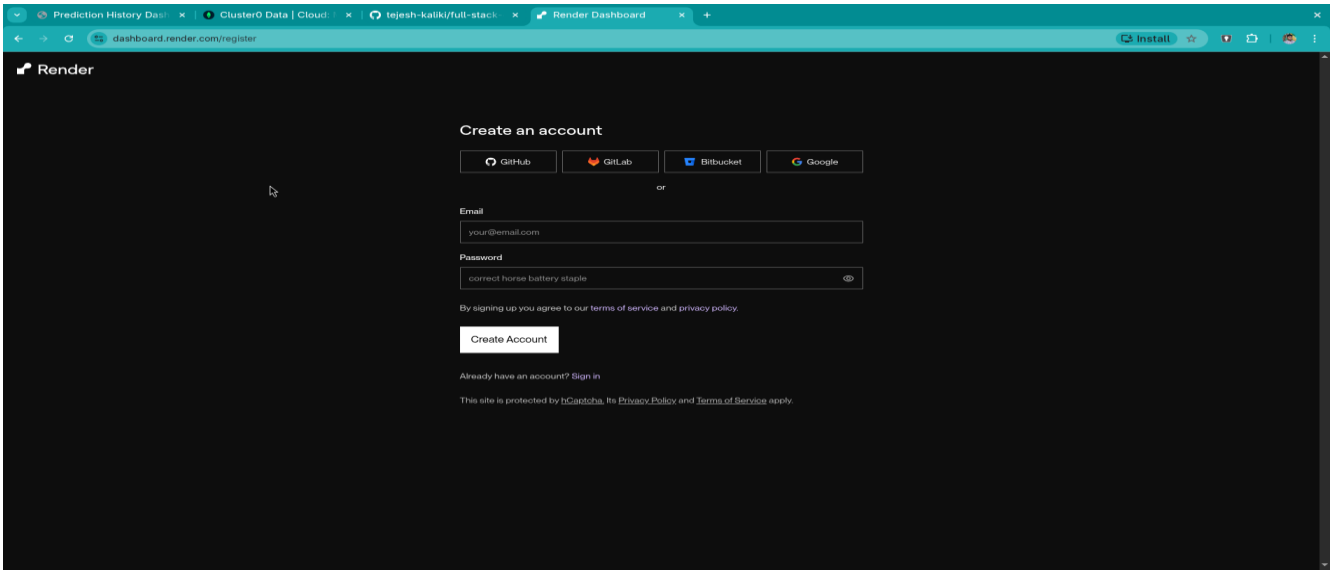
After creating repository



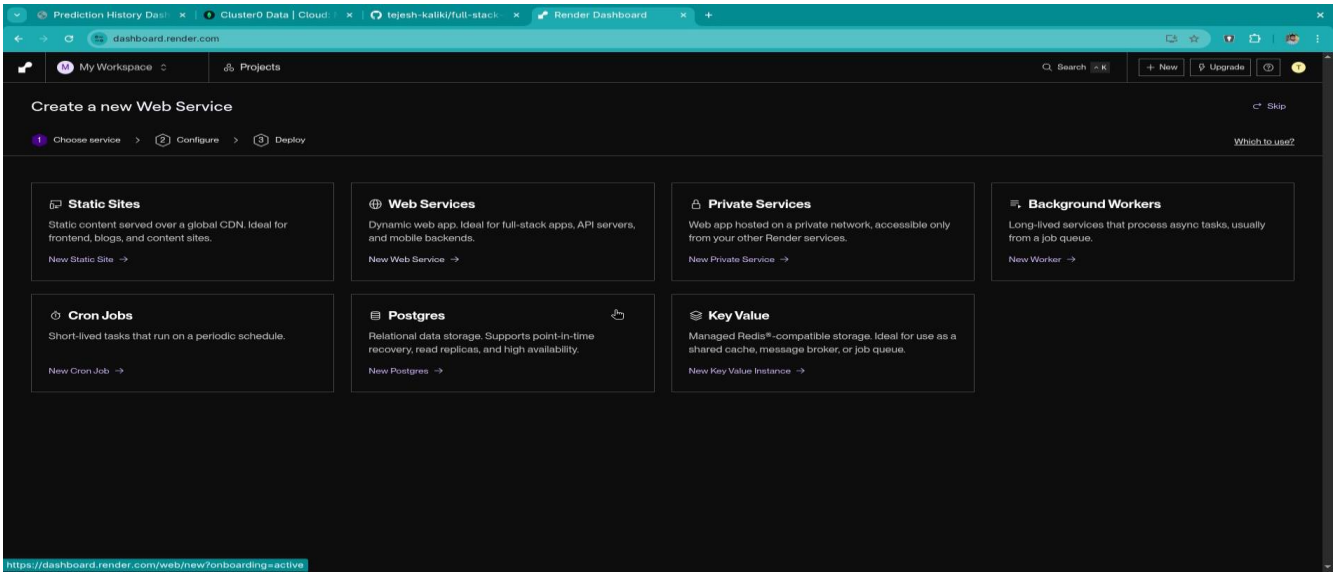
After initial commit

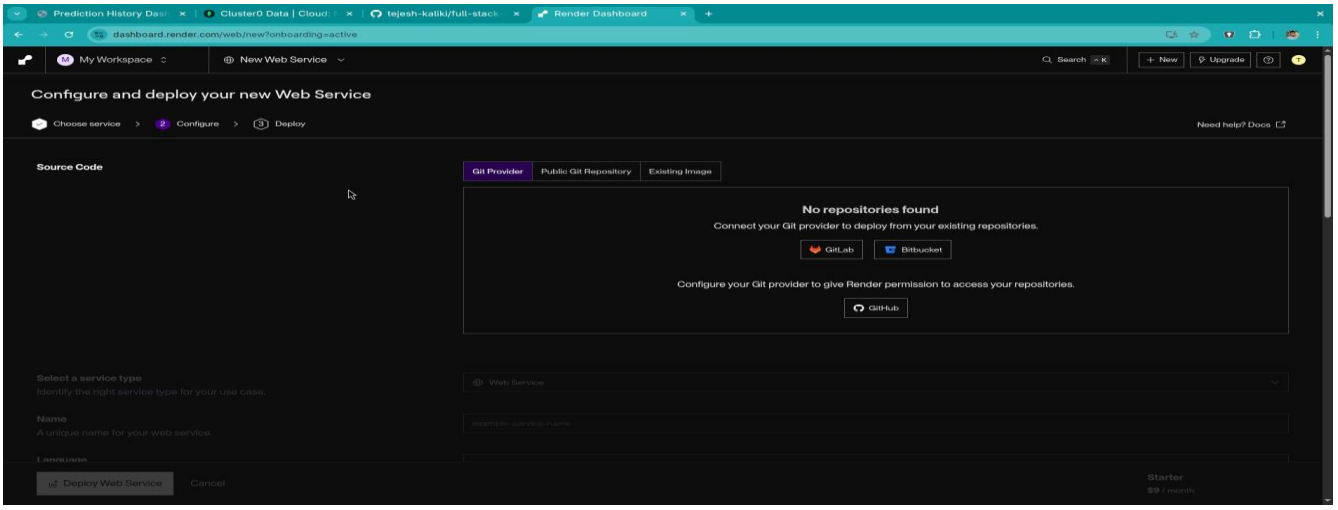


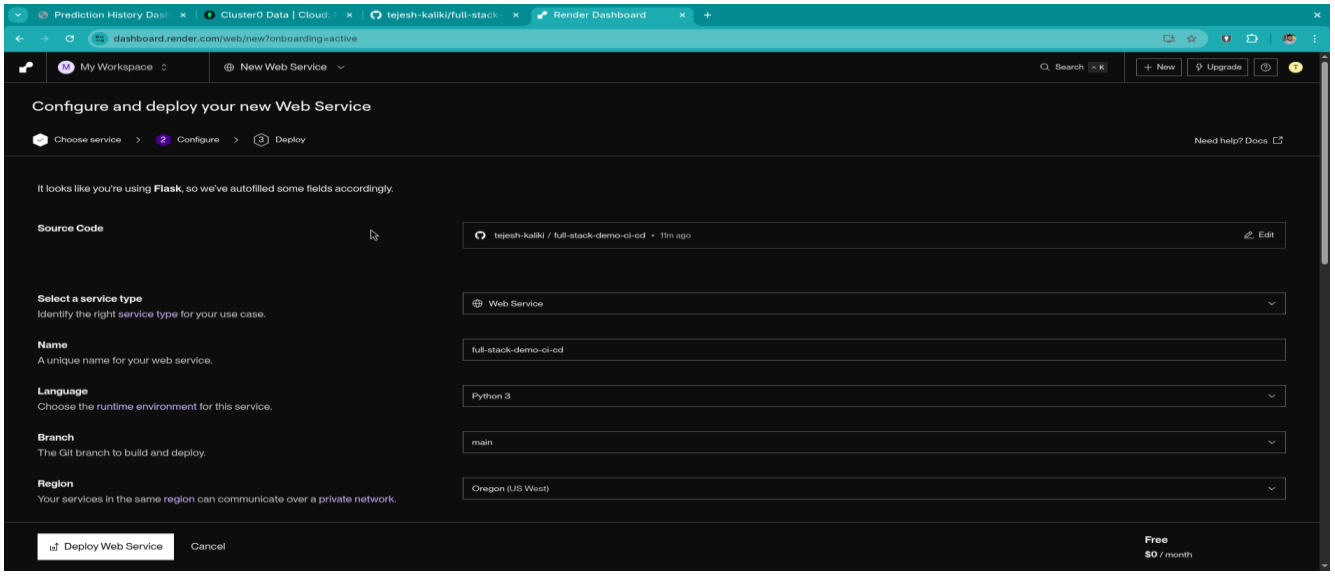
registration

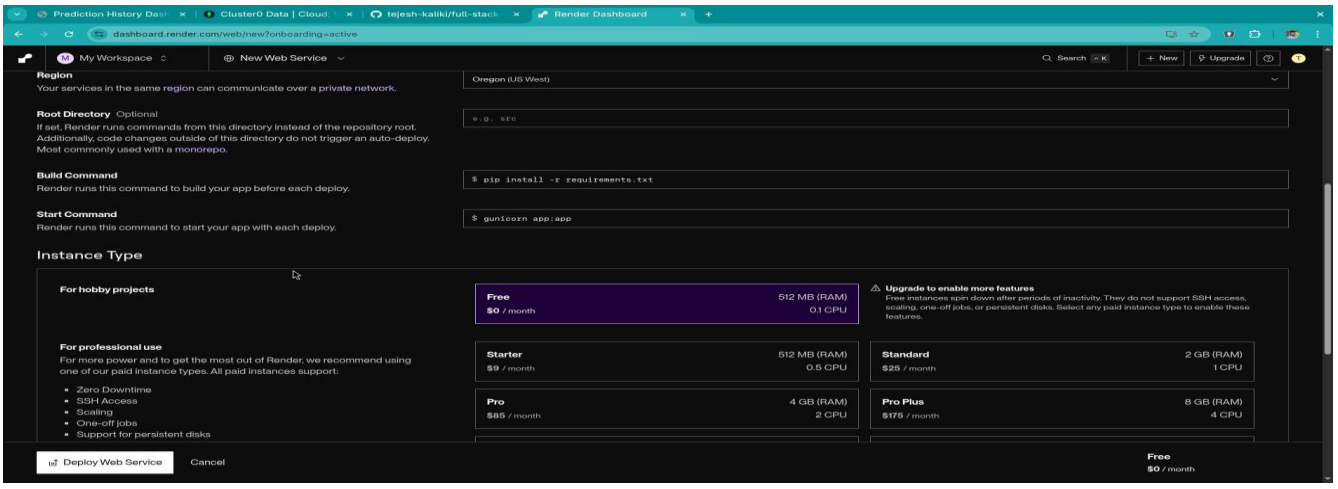


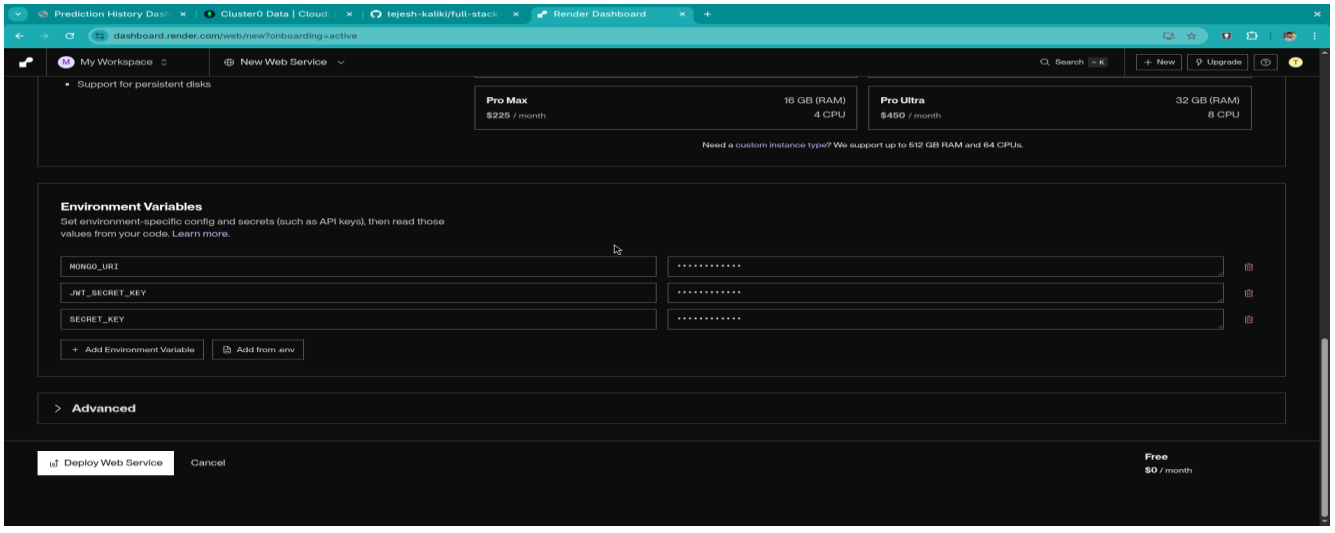
render.com service creation



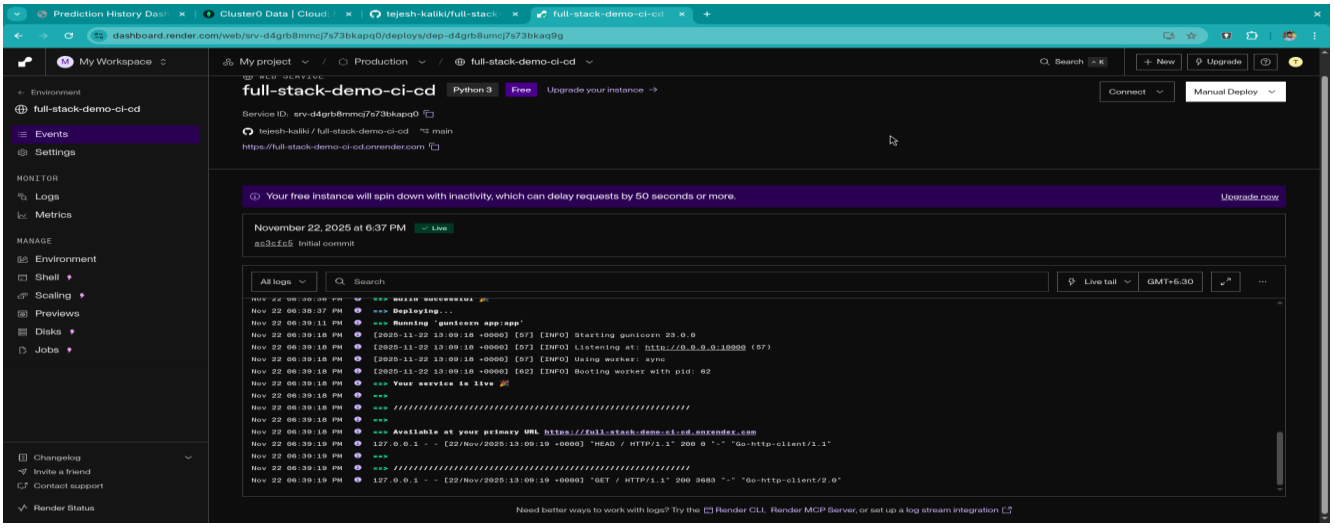


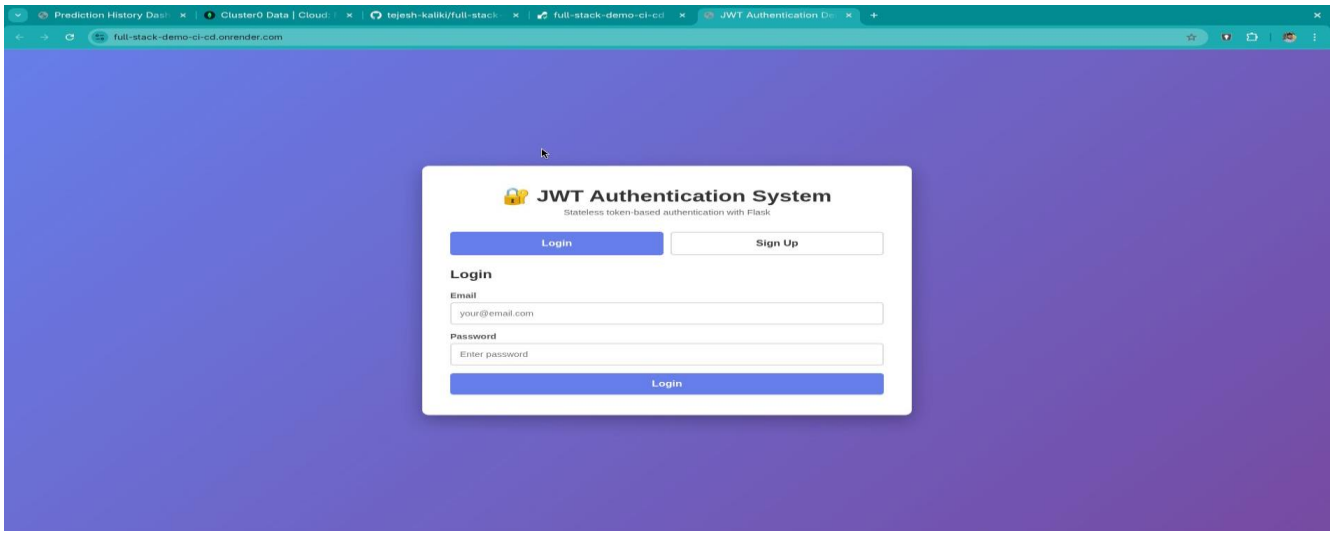




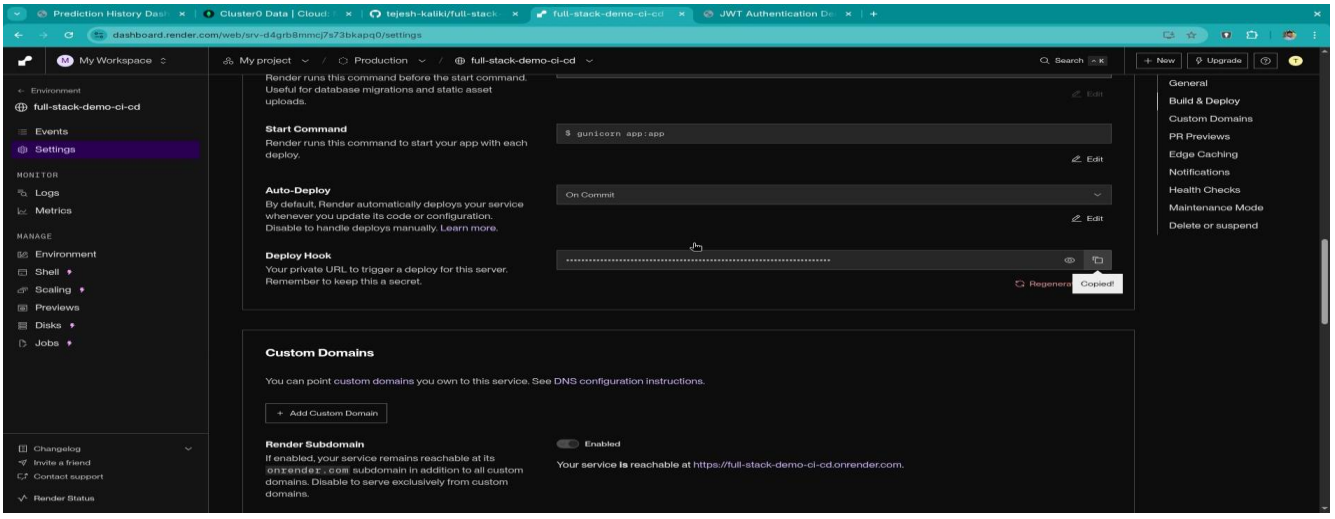


Successful deployment to `render.com`

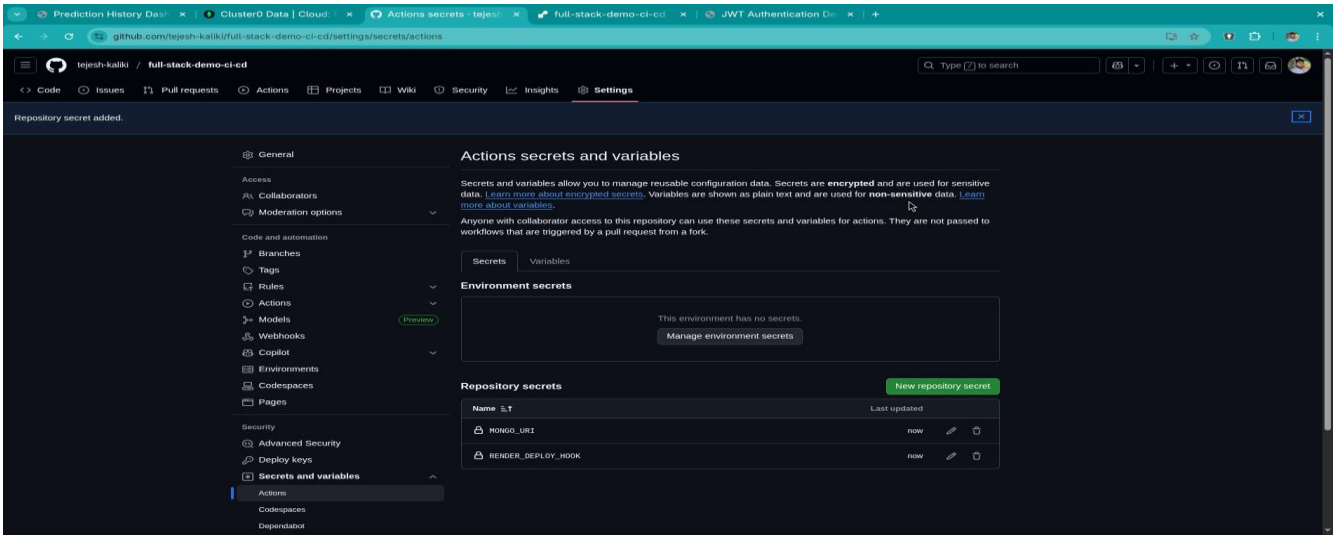




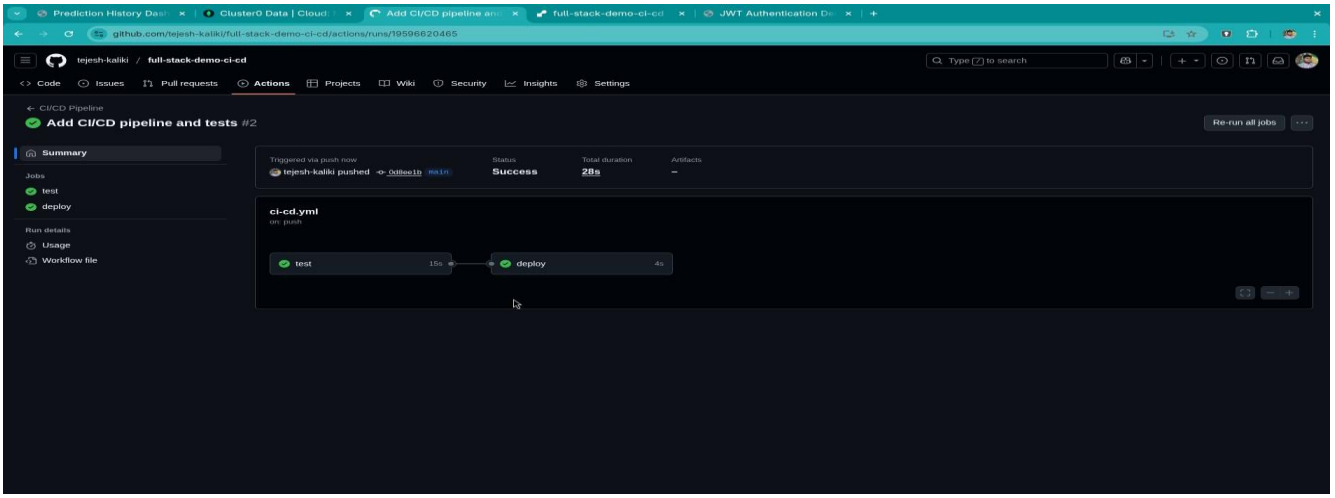
`render.com` Deploy Webhook



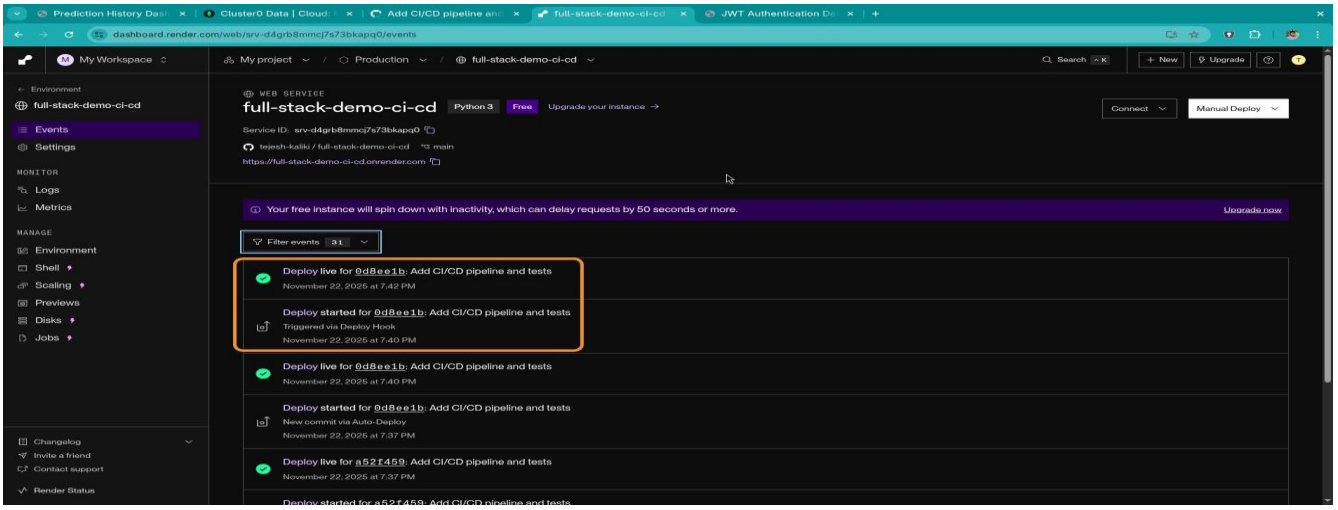
GitHub Secrets Setup



GitHub Workflow Execution success



Successful re-deploy after webhook (Check the highlighted part showing "Triggerred via Deploy Webhook")



Result

Successfully implemented automated CI/CD pipeline using GitHub Actions and Render, demonstrating modern DevOps practices where code changes automatically trigger tests and deployment to production.

> **Caution**
>
> **VERY IMPORTANT:** Please delete the service in `render.com` once the experiment is done, and make sure to select ***free plan*** when creating.

Experiment 11: Build Mini-Project: Spam Detector with Text Classification

Aim

To build a complete mini-project that implements a spam/ham detector using machine learning, combining all previous concepts (ML training, API, frontend, database, authentication).

Software Requirements

- Python 3.10+
- Flask
- scikit-learn
- pymongo
- MongoDB Atlas
- pandas
- numpy

Theory

This mini-project demonstrates a real-world ML application: SMS/Email spam classification. We'll use the SMS Spam Collection dataset (5,574 SMS messages labeled as spam or ham). The pipeline includes: (1) data preprocessing (tokenization, lowercasing, special character removal), (2) feature extraction using TF-IDF vectorization, (3) model training with Naive Bayes, (4) API deployment, (5) user authentication, and (6) storing predictions in MongoDB. This showcases an end-to-end ML workflow suitable for production.

Procedure

Step 1: Create Project Directory

```
mkdir exp11_spam_detector cd
exp11_spam_detector
```

Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

Step 3: Install Required Packages

```
pip install flask scikit-learn pymongo python-dotenv pandas numpy bcrypt
```

Step 4: Create .env File

> **Note**
>
> Replace `MONGO_U` with actual URI from previous steps.

```
MONGO_URI=mongodb+srv://admin:YOUR_PASSWORD@cluster0.xxxxx.mongodb.net/  SECRET_KEY=your_secret_key_here
```

Step 5: Create Project Structure

```
mkdir templates mkdir
static mkdir static/css
mkdir static/js mkdir
models
```

Step 6: Download and Prepare Dataset

Create `prepare_data.py`:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix
import pickle
import os

# Download SMS Spam Collection Dataset
print("Downloading SMS Spam Collection
dataset...")
url = "https://raw.githubusercontent.com/justmarkham/talkingdata-mobile-user-
demographics/master/data/sms_spam_collection.txt"

try:
    df = pd.read_csv(url, sep='\t', header=None, names=['label', 'message']) except:
    print("Using sample data instead...")
    # Create sample dataset if download fails df
    = pd.DataFrame({
        'label': ['ham', 'spam', 'ham', 'ham', 'spam'] * 100, 'message': [
            'Hey, how are you?',
            'WINNER! Claim your prize NOW!!!',
            'Meeting at 3pm today',
            'Call me when you arrive',
            'Click here for FREE money'
        ] * 100
    })

print(f"Dataset loaded: {len(df)} messages")
print(f"\nLabel distribution:\n{df['label'].value_counts()}")
```

```
# Prepare data
X = df['message']
y = df['label'].map({'ham': 0, 'spam': 1})

# Split data
X_train, X_test, y_train, y_test = train_test_split( X, y,
    test_size=0.2, random_state=42, stratify=y
)

print(f"\nTraining set: {len(X_train)} messages")
print(f"Testing set: {len(X_test)} messages")

# Create pipeline
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=1000, lowercase=True,
                              stop_words='english', ngram_range=(1, 2))),
    ('clf', MultinomialNB())
])

# Train model
print("\nTraining Naive Bayes classifier...")
pipeline.fit(X_train, y_train)

# Evaluate
y_pred = pipeline.predict(X_test) accuracy
= accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"\nModel Performance:")
print(f"Accuracy:  {accuracy * 100:.2f}%")
print(f"Precision: {precision * 100:.2f}%")
print(f"Recall:    {recall * 100:.2f}%")
print(f"F1-Score:  {f1 * 100:.2f}%")

print(f"\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Save model
os.makedirs('models', exist_ok=True)
with open('models/spam_detector.pkl', 'wb') as f: pickle.dump(pipeline,
```

```
    f)

print("\nModel saved to models/spam_detector.pkl")

# Test samples print("\n"
+ "="*60) print("Test
Predictions")
print("="*60)

test_messages = [
    "Hey, how are you doing today?",
    "CLICK HERE NOW!!! WIN FREE
    IPHONE!!!",
    "Meeting scheduled for tomorrow at 2pm",
    "Congratulations! You've won $1,000,000!"
]

for msg in test_messages:
    pred = pipeline.predict([msg])[0]
    prob = pipeline.predict_proba([msg])[0]
```

```
    label = 'SPAM' if pred == 1 else 'HAM' confidence = max(prob)
    * 100 print(f"\nMessage: {msg}")
    print(f"Prediction: {label} (Confidence: {confidence:.2f}%)")
```

Step 7: Run Data Preparation

```
python prepare_data.py
```

Step 8: Create app.py

```
from flask import Flask, render_template, request, jsonify, session from
pymongo import MongoClient
import pickle
import os
from dotenv import load_dotenv
from datetime import datetime
import bcrypt

load_dotenv()

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY')
```

```
# MongoDB connection
client =
MongoClient(os.getenv('MONGO_URI')) db =
client['fullstack_ai_lab'] users_collection =
db['users'] predictions_collection =
db['spam_predictions']

# Load model
with open('models/spam_detector.pkl', 'rb') as f:
    spam_detector = pickle.load(f)

print("Spam detector model loaded successfully!")

def require_login(f):
    """Decorator to require login""" def
    wrapper(*args, **kwargs):
        if 'user' not in session:
            return jsonify({'error': 'Not authenticated'}), 401
        return f(*args, **kwargs)
    wrapper.__name_ = f.__name_
    return wrapper

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/api/register', methods=['POST'])
def register():
    """Register new user"""
    try:
        data = request.get_json() email
        = data.get('email') password =
        data.get('password')

        if not email or not password:
            return jsonify({'error': 'Email and password required'}), 400

        if users_collection.find_one({'email': email}):
            return jsonify({'error': 'Email already exists'}), 400

        hashed = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
        users_collection.insert_one({'email': email, 'password': hashed})
```

```
        return jsonify({'message': 'Registration successful'}), 201 except
    Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/api/login', methods=['POST'])
def login():
    """Login user"""
    try:
        data = request.get_json() email
        = data.get('email') password =
        data.get('password')

        user = users_collection.find_one({'email': email})

        if not user or not bcrypt.checkpw(password.encode('utf-8'), user['password']):
            return jsonify({'error': 'Invalid credentials'}), 401

        session['user'] = email
        return jsonify({'message': 'Login successful', 'user': email}), 200 except
    Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/api/logout', methods=['POST'])
def logout():
    """Logout user"""
    session.pop('user', None)
    return jsonify({'message': 'Logged out'}), 200

@app.route('/api/check-auth', methods=['GET'])
def check_auth():
    """Check if user is logged in""" if
    'user' in session:
        return jsonify({'authenticated': True, 'user': session['user']}), 200 return
    jsonify({'authenticated': False}), 200

@app.route('/api/detect-spam', methods=['POST'])
@require_login
def detect_spam():
    """Detect spam in message"""
```

```
    try:
        data = request.get_json()
```

```
        message = data.get('message', '').strip()

        if not message:
            return jsonify({'error': 'Message required'}), 400

        if len(message) < 3:
            return jsonify({'error': 'Message too short'}), 400

        # Make prediction
        prediction = spam_detector.predict([message])[0]
        probabilities = spam_detector.predict_proba([message])[0]

        label = 'SPAM' if prediction == 1 else 'HAM'
        spam_confidence = float(probabilities[1])
        ham_confidence = float(probabilities[0])

        # Store prediction
        prediction_record = {
            'user': session.get('user'),
            'timestamp': datetime.utcnow(),
            'message': message, 'prediction':
            label,
            'spam_confidence':  spam_confidence,
            'ham_confidence': ham_confidence,
            'message_length': len(message),
            'word_count': len(message.split())
        }

        result = predictions_collection.insert_one(prediction_record) return

        jsonify({
            'prediction': label, 'spam_confidence':
            spam_confidence, 'ham_confidence':
            ham_confidence, 'record_id':
            str(result.inserted_id)
        }), 200

    except Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/api/history', methods=['GET'])
@require_login
def get_history():
```

```
    """Get user's prediction history"""
    try:
        user = session.get('user')
        limit = int(request.args.get('limit', 20))

        predictions = list(
            predictions_collection.find({'user': user})
            .sort('timestamp', -1)
            .limit(limit)
        )

        # Convert ObjectId to string for JSON
        for pred in predictions:
            pred['_id'] = str(pred['_id'])
            pred['timestamp'] = pred['timestamp'].isoformat()

        return jsonify(predictions), 200 except
    Exception as e:
        return jsonify({'error': str(e)}), 500

@app.route('/api/stats', methods=['GET'])
@require_login
def get_stats():
    """Get user's statistics""" try:
        user = session.get('user')

        predictions = list(predictions_collection.find({'user': user})) total =

        len(predictions)
        spam_count = sum(1 for p in predictions if p['prediction'] == 'SPAM')
        ham_count = total - spam_count

        avg_spam_conf = sum(p['spam_confidence'] for p in predictions) / total if total > 0
else 0
        avg_message_length = sum(p['message_length'] for p in predictions) / total if total
```

```
> 0 else 0

        return jsonify({
            'total_messages': total,
            'spam_count': spam_count,
            'ham_count': ham_count,
            'spam_percentage': (spam_count / total * 100) if total > 0 else 0,
            'avg_spam_confidence': avg_spam_conf,
            'avg_message_length':  avg_message_length
        }), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True,  host='0.0.0.0',  port=5000)
```

Step 9: Create templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta  charset="UTF-8">
    <meta  name="viewport"  content="width=device-width,  initial-scale=1.0">
    <title>Spam Detector - Full Stack AI</title>
    <link  rel="stylesheet"  href="{{  url_for('static',  filename='css/spam.css')  }}">
```

```
    <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js">
</script>
</head>
<body>
    <div class="container">
        <header>
            <h1>⍰  Spam  Detector</h1>
            <nav id="nav"></nav>
        </header>

        <main id="main"></main>
    </div>

    <script  src="{{  url_for('static',  filename='js/spam.js')  }}"></script>
</body>
</html>
```

Step 10: Create static/css/spam.css (comprehensive styling)

Due to space constraints, here's a minimal version:

```
* {
    margin: 0;
    padding: 0;
    box-sizing:  border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #667eea 0%, #764ba2
    100%); min-height: 100vh;
}

.container {
    max-width: 1000px;
    margin: 0 auto;
    padding: 20px;
}

header {
    background: white;
    padding: 20px;
    border-radius: 10px;
    margin-bottom:
    30px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}

h1 {
    color: #333;
    margin-bottom:  15px;
}
nav {
    display: flex; gap:
    10px;
}

.btn {
    padding: 10px 20px;
    background:
    #667eea; color:
```

```
    white; border: none;
    border-radius: 5px;
    cursor: pointer;
}

.btn:hover {
    background: #764ba2;
}

main {
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 5px 15px rgba(0,0,0,0.1);
}

.form-group {
    margin-bottom:  20px;
}

label {
    display: block;
    margin-bottom:
    5px; font-weight:
    600;
}

input, textarea {
    width: 100%;
    padding: 12px;
    border: 2px solid #ddd;
    border-radius: 5px;
    font-size: 14px;
}

textarea {
    resize: vertical;
    min-height: 100px;
}

input:focus, textarea:focus {
    outline: none;
    border-color:  #667eea;
```

```
}

.prediction-card {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    color: white;
    padding: 30px;
    border-radius: 10px;
    text-align: center;
    margin-top: 20px;
}

.prediction-label {
    font-size: 32px;
    font-weight: bold;
    margin-bottom: 15px;
}

.confidence {
    display: flex;
    justify-content: space-around;
    margin-top: 20px;
}

.confidence-bar {
    flex: 1;
    margin: 0 10px;
}

.bar {
    width: 100%;
    height: 30px;
    background: rgba(255,255,255,0.3);
    border-radius: 5px;
    overflow: hidden;
}

.bar-fill {
    height: 100%;
    background: rgba(255,255,255,0.8);
    transition: width 0.5s;
}
```

Step 11: Create static/js/spam.js

```
const BASE_URL = ''; let
spamChart = null;

async function checkAuth() { try {
        const response = await fetch(`${BASE_URL}/api/check-auth`); const data = await
        response.json(); renderUI(data.authenticated);
    } catch (error) {
        console.error('Auth check failed:', error);
    }
}
```

```
function renderUI(authenticated) {
    const nav = document.getElementById('nav');
    const main = document.getElementById('main');

    if (!authenticated) {
        nav.innerHTML = '';
        main.innerHTML = `
            <div id="loginSection">
                <h2>Login / Register</h2>
                <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 30px;">
                    <div>
                        <h3>Login</h3>
                        <form id="loginForm">
                            <div class="form-group">
                                <label>Email</label>
                                <input type="email" id="loginEmail" required>
                            </div>
                            <div class="form-group">
                                <label>Password</label>
                                <input type="password" id="loginPassword" required>
                            </div>
                            <button type="submit" class="btn">Login</button>
                        </form>
                    </div>
                    <div>
                        <h3>Register</h3>
                        <form id="registerForm">
                            <div class="form-group">
                                <label>Email</label>
                                <input type="email" id="regEmail" required>
```

```
                    </div>
                    <div class="form-group">
                        <label>Password</label>
                        <input type="password" id="regPassword" required>
                    </div>
                    <button type="submit" class="btn">Register</button>
                </form>
            </div>
        </div>
        <div id="message" style="margin-top: 20px; padding: 15px; border-radius:
5px; display: none;"></div>
    </div>
`;

    document.getElementById('loginForm').addEventListener('submit', handleLogin);
    document.getElementById('registerForm').addEventListener('submit', handleRegister);
} else {
    nav.innerHTML = `<button class="btn" onclick="logout()">Logout</button>`;
    main.innerHTML = `
        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 20px;">
            <div id="detectorSection">
             <h2>Check Message</h2>
      <form id="spamForm">
          <div class="form-group">
              <label for="message">Enter message to check:</label>
              <textarea id="message" placeholder="Paste message here..."

  required></textarea>

          </div>
          <button type="submit" class="btn" style="width: 100%;">Detect


   Spam</button>

          </form>
          <div id="result"></div>

 </div>
            <div id="statsSection">
                <h2>Your Statistics</h2>
                <div id="stats"></div>
                <div id="chart-container" style="margin-top: 20px;">
                    <canvas id="spamChart"></canvas>
                </div>
            </div>
```

```
                </div>
                <div id="historySection" style="margin-top: 30px;">
                    <h2>Recent Predictions</h2>
                    <div id="history"></div>
                </div>
            `;

            document.getElementById('spamForm').addEventListener('submit', handleSpamDetection);
            loadStats();
            loadHistory();
        }
}

async function handleLogin(e) {
    e.preventDefault();
    const email = document.getElementById('loginEmail').value;
    const password =
    document.getElementById('loginPassword').value;

    try {
        const response = await fetch(`${BASE_URL}/api/login`, {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({email, password})
        });

        const data = await response.json(); if

        (response.ok) {
            showMessage('Login successful!', 'success'); setTimeout(()
            => checkAuth(), 1000);
        } else {
            showMessage(data.error || 'Login failed', 'error');
        }
    } catch (error) {
        showMessage('Error: ' + error.message, 'error');

}
}

async function handleRegister(e) {
    e.preventDefault();
    const email = document.getElementById('regEmail').value;
    const password =
```

```
    document.getElementById('regPassword').value;

    try {
        const response = await fetch(`${BASE_URL}/api/register`, {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({email, password})
        });

        const data = await response.json(); if

        (response.ok) {
                showMessage('Registration successful! Please login.', 'success');
         } else {
                 showMessage(data.error || 'Registration failed', 'error');
        }
    } catch (error) {
        showMessage('Error: ' + error.message, 'error');
    }
}

async function handleSpamDetection(e)
    { e.preventDefault();
    const message = document.getElementById('message').value;

    try {
        const response = await fetch(`${BASE_URL}/api/detect-spam`, {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({message})
        });

        const data = await response.json(); if

        (response.ok) {
            const resultDiv = document.getElementById('result'); const
            spamConf = (data.spam_confidence * 100).toFixed(2); const
            hamConf = (data.ham_confidence * 100).toFixed(2);

            resultDiv.innerHTML = `
                <div class="prediction-card">
                    <div class="prediction-label">${data.prediction}</div>
                    <div class="confidence">
                        <div class="confidence-bar">
                            <label>SPAM: ${spamConf}%</label>
```

```
                        <div class="bar">
                            <div class="bar-fill" style="width: ${spamConf}%"></div>
                        </div>
                    </div>
                    <div  class="confidence-bar">
                        <label>HAM: ${hamConf}%</label>
                        <div class="bar">
                            <div class="bar-fill" style="width: ${hamConf}%"></div>
                        </div>
                    </div>
                </div>
            </div>
        `;

        loadStats();
        loadHistory();
    } else {
        showMessage(data.error || 'Detection failed', 'error');
    }
} catch (error) {
    showMessage('Error: ' + error.message, 'error');
}
}

async function loadStats() {
    try {
        const response = await fetch(`${BASE_URL}/api/stats`);
        const data = await response.json();

        if (response.ok) {
            const statsDiv = document.getElementById('stats');
            statsDiv.innerHTML = `
                <div style="background: #f5f5f5; padding: 15px; border-radius: 5px;">
                    <p><strong>Total  Messages:</strong>  ${data.total_messages}</p>
                    <p><strong>Spam:</strong> ${data.spam_count}
(${data.spam_percentage.toFixed(1)}%)</p>
                    <p><strong>Ham:</strong>  ${data.ham_count}</p>
                    <p><strong>Avg Spam Confidence:</strong> ${(data.avg_spam_confidence *
100).toFixed(2)}%</p>
                    <p><strong>Avg Message Length:</strong>
${data.avg_message_length.toFixed(0)}  chars</p>
                </div>
            `;
```

```
            // Draw chart
            if (data.total_messages > 0) {
                const ctx = document.getElementById('spamChart').getContext('2d');

                if (spamChart) spamChart.destroy();

                spamChart = new Chart(ctx, {
                    type: 'doughnut',
                    data: {
                        labels: ['Spam', 'Ham'],
                        datasets: [{
                        data: [data.spam_count, data.ham_count],
                        backgroundColor: ['#ff6b6b', '#4ecdc4']
                    }]

                    }

                });
            }

        }

    catch (error) {
        console.error('Failed to load stats:', error);
    }
}

async function loadHistory() { try
    {
        const response = await fetch(`${BASE_URL}/api/history?limit=5`);
        const predictions = await response.json();

        if (response.ok && predictions.length > 0) {
            const historyDiv = document.getElementById('history');
            historyDiv.innerHTML = predictions.map(p => `
                <div style="background: #f5f5f5; padding: 15px; margin-bottom: 10px; border-
radius: 5px; border-left: 4px solid ${p.prediction === 'SPAM' ? '#ff6b6b' : '#4ecdc4'};">
                    <p><strong>${p.prediction}</strong> - ${(p.spam_confidence *
100).toFixed(2)}% confidence</p>
                    <p style="color: #666; font-size: 12px;">"${p.message.substring(0,
50)}..."</p>
                    <p style="color: #999; font-size: 12px;">${new
Date(p.timestamp).toLocaleString()}</p>
```

```
                </div>
            `).join('');
        }
    } catch (error) {
        console.error('Failed to load history:', error);
    }
}

async function logout() { try
    {
        await fetch(`${BASE_URL}/api/logout`, {method: 'POST'});
        checkAuth();
    } catch (error) {
        console.error('Logout failed:', error);
    }
}

function showMessage(text, type) {
    const messageDiv =
    document.getElementById('message'); if (messageDiv) {
        messageDiv.textContent = text;
        messageDiv.style.background = type === 'success' ? '#d4edda' : '#f8d7da';
        messageDiv.style.color = type === 'success' ? '#155724' : '#721c24';
        messageDiv.style.display = 'block';
        setTimeout(() => {
```

```
                messageDiv.style.display = 'none';
            }, 5000);
        }
    }

    // Initialize checkAuth();
```

Step 12: Run the Application

```
python app.py
```

Step 13: Test the Spam Detector

1. Open `http://localhost:5000`
2. Register a new account
3. Login with credentials
4. Enter test messages:
    - Ham: "Hi, how are you doing today?"

- Spam: "CLICK NOW!!! WIN FREE MONEY!!!"

5. View predictions with confidence scores
6. Check statistics and prediction history

Output

A complete spam detection mini-project with:

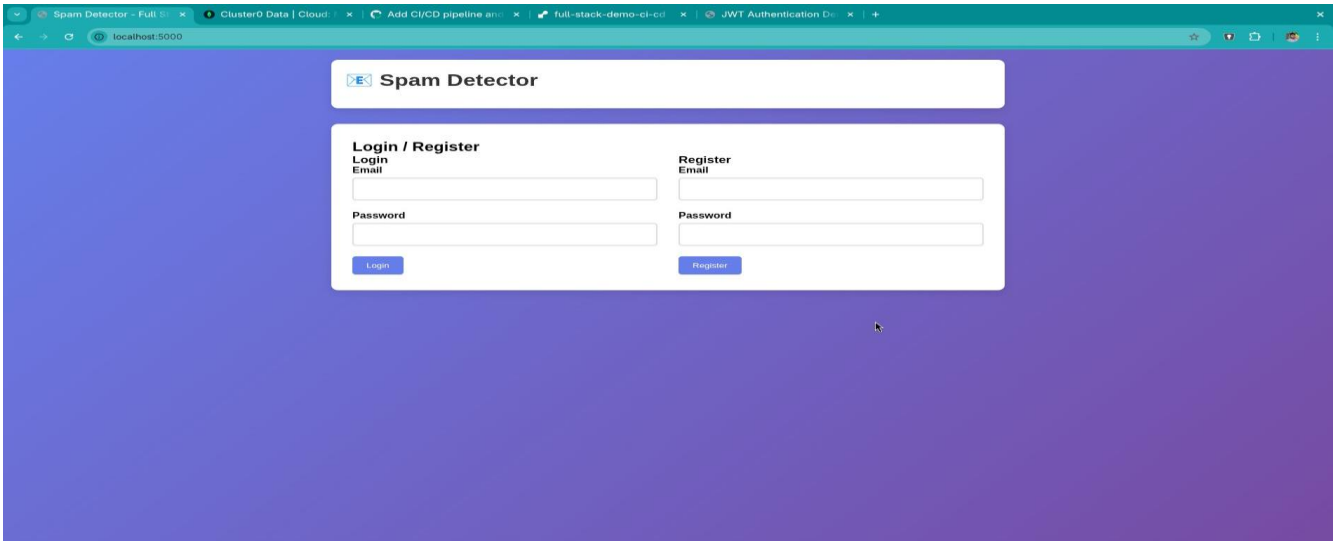
- User authentication system
- Real-time spam detection
- Confidence scoring (spam % vs ham %)
- User statistics dashboard
- Prediction history tracking
- Chat-like interface
- Chart visualization of spam distribution
- Real-world SMS dataset
- TF-IDF feature extraction
- Naive Bayes classifier

Screen-shots

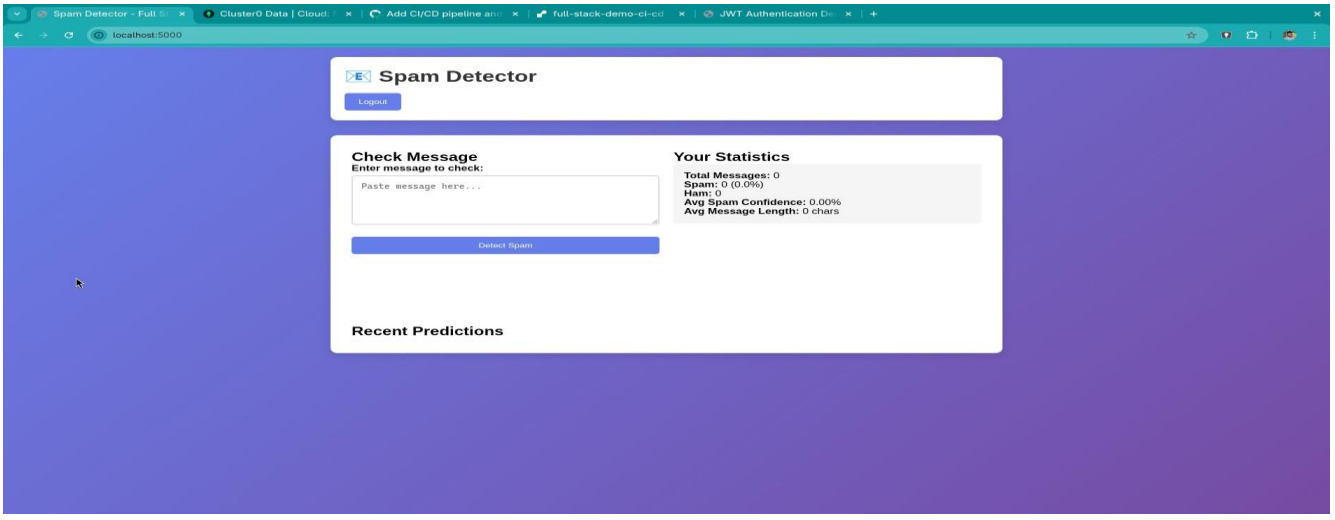
Output of `prepare_data.py`:

```
personal/lab_manual/exp11_spam_detector 2s
venv > python prepare_data.py
Downloading SMS Spam Collection dataset...
Using sample data instead...
Dataset loaded: 500 messages

Label distribution:
label
ham     300
spam    200
Name: count, dtype: int64

Training set: 400 messages
Testing set: 100 messages

Training Naive Bayes classifier...

Model Performance:
Accuracy:  100.00%
Precision: 100.00%
Recall:    100.00%
F1-Score:  100.00%

Confusion Matrix:
[[60  0]
 [ 0 40]]

Model saved to models/spam_detector.pkl

============================================================
Test Predictions
============================================================

Message: Hey, how are you doing today?
Prediction: HAM (Confidence: 99.78%)

Message: CLICK HERE NOW!!! WIN FREE IPHONE!!!
Prediction: SPAM (Confidence: 98.98%)

Message: Meeting scheduled for tomorrow at 2pm
Prediction: HAM (Confidence: 98.18%)

Message: Congratulations! You've won $1,000,000!
Prediction: HAM (Confidence: 60.00%)
```

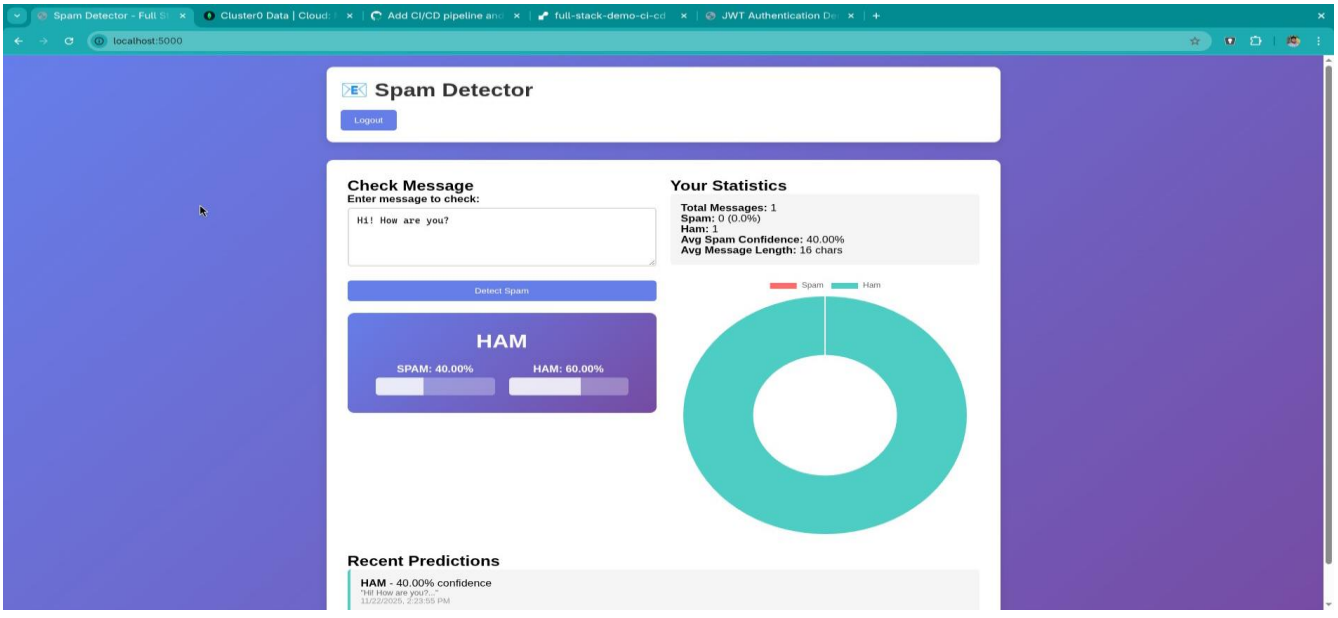
Default Home Page:



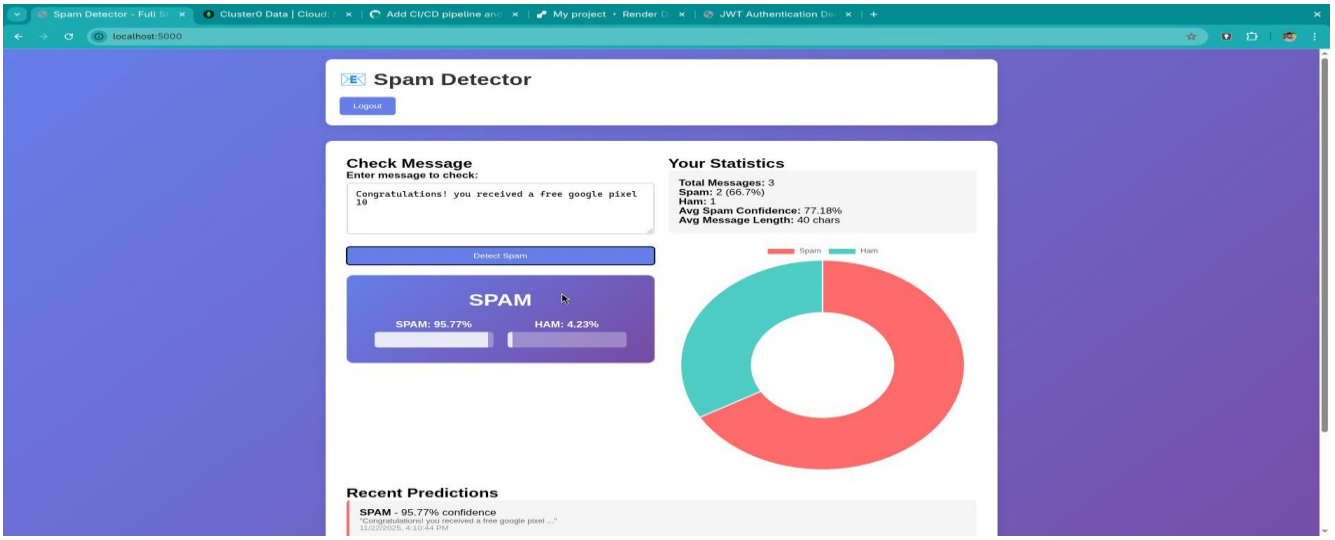
After successful login:



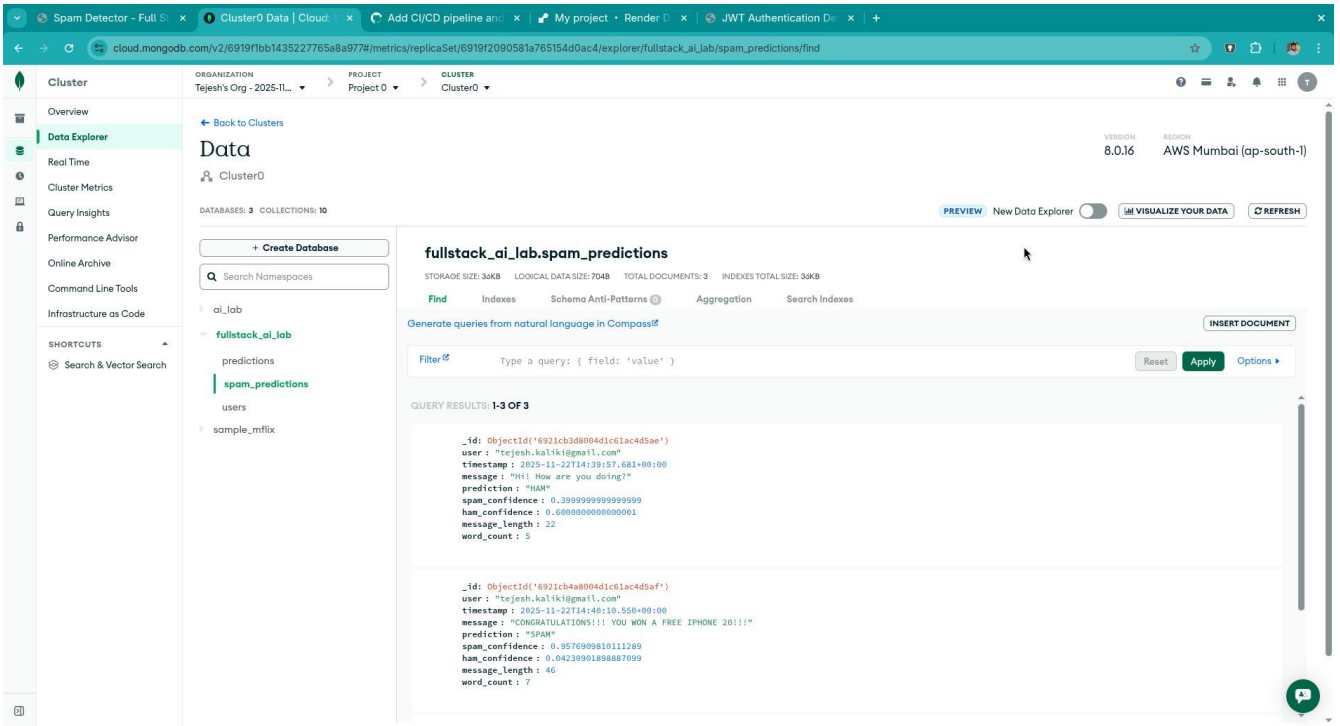
Successful Ham Detection



Successful Spam Detection



Data stored in MongoDB



Result

Successfully built and deployed a complete mini-project demonstrating end-to-end ML application development with user authentication, database storage, and interactive UI.

Experiment 12: Final Demo & Deployment on Render

Aim

To deploy the complete full-stack AI application to Render cloud platform, making it accessible from anywhere with a public URL.

Software Requirements

- Flask application (any from previous experiments)
- GitHub account with repository
- Render.com account
- Git
- Internet connection

Theory

Cloud deployment makes applications globally accessible and scalable. Render is a cloud platform offering free hosting for web applications. The deployment process involves: (1) preparing the application with proper configuration files, (2) pushing code to GitHub, (3) connecting the GitHub repository to Render, (4) setting environment variables, (5) configuring build and start commands, and (6) monitoring the live application. This demonstrates production-grade deployment suitable for real-world applications.

Procedure

Step 0: Copy the previous Spam detector code, or any other project as `exp12_final_project`

Step 1: Prepare Your Flask Application

Ensure your main application directory contains:

- `app.py` - Main Flask application
- `requirements.txt` - All Python dependencies
- `Procfile` - Deployment instructions
- `templates/` - Flask templates
- `static/` - CSS, JS, images
- `.env.example` - Example environment variables (don't commit .env)

Step 2: Create Procfile

Create `Procfile` in root directory:

```
web: gunicorn app:app
```

Step 3: Update requirements.txt

Ensure `gunicorn` is included:

```
pip install gunicorn
pip freeze > requirements.txt
```

Step 4: Create Runtime Configuration

Create `runtime.txt`:

```
python-3.10.13
```

Step 5: Configure for Production

Update `app.py` for production:

```
# --- Do not modify any code above this ---

# Replace the block at end of code with this if __name____ == '__
main__':
    debug = os.getenv('FLASK_DEBUG', 'False') == 'True' app.run(debug=debug, host='0.0.0.0',
    port=int(os.getenv('PORT', 5000)))
```

Step 6: Create .gitignore

Create `.gitignore` file:

```
venv/
__pycache__/
*.pyc
.env
.DS_Store uploads/
```

Step 7: Initialize and push to GitHub

```
git init git add .
git commit -m "Initial commit" git branch -M
main

# Modify the url at the end of this
git remote add origin https://github.com/YOUR_USERNAME/fullstack-ai-lab.git

git push -u origin main
```

Step 8: Create Render Account

> **Important**
>
> If account is already created for experiment 10, just login back.

1. Go to https://render.com
2. Sign up with GitHub account
3. Click "Connect GitHub"
4. Authorize Render to access your repositories

Step 9: Deploy to Render

1. Go to https://dashboard.render.com

2. Click "New +" → "Web Service"
3. Select your `fullstack-ai-lab` repository
4. Configure deployment:
    - Name: `fullstack-ai-lab` (or unique name)
    - Environment: `Python 3`
    - Build Command: `pip install -r requirements.txt`
    - Start Command: `gunicorn app:app`
    - Plan: "Free"
5. Click "Create Web Service"

Step 10: Set Environment Variables

In Render dashboard:

1. Go to your service settings
2. Scroll to "Environment"
3. Add variables:
    - `MONGO_URI`: Your MongoDB connection string
    - `JWT_SECRET_KEY`: Your secret key
    - `SECRET_KEY`: Your secret key
    - `FLASK_DEBUG` `False` (for production)

Step 11: Monitor Deployment

1. Go to your service
2. Click "Events" tab
3. Watch the deployment logs
4. Wait for "Your service is live" message
5. Click the service URL to access your app

Step 12: Test Live Application

Your application is now live at: `https://your-service-name.onrender.com`

Test all features:

- Authentication (login/signup)
- ML predictions
- Database operations
- File uploads (if applicable)
- API endpoints

Step 13: Enable Auto-Deployments (Optional)

1. In Render dashboard → Service Settings
2. Enable "Auto-Deploy" when commits are pushed to GitHub
3. Choose branch: `main`

Step 14: Monitor Application Health

1. Check logs regularly: Dashboard → Events
2. Monitor resource usage
3. Set up alerts if available
4. Keep dependencies updated

Deployment Checklist

Before deploying, verify:

- ✅ All code committed to GitHub
- ✅ `requirements.txt` includes all dependencies
- ✅ `Procfile` created

- ✅ MongoDB Atlas allows Render IP (usually auto-whitelisted)
- ✅No hardcoded secrets in code
- ✅ `.env` file in `.gitignore`
- ✅Application runs locally first
- ✅All environment variables documented
- ✅Database migrations (if any) handled

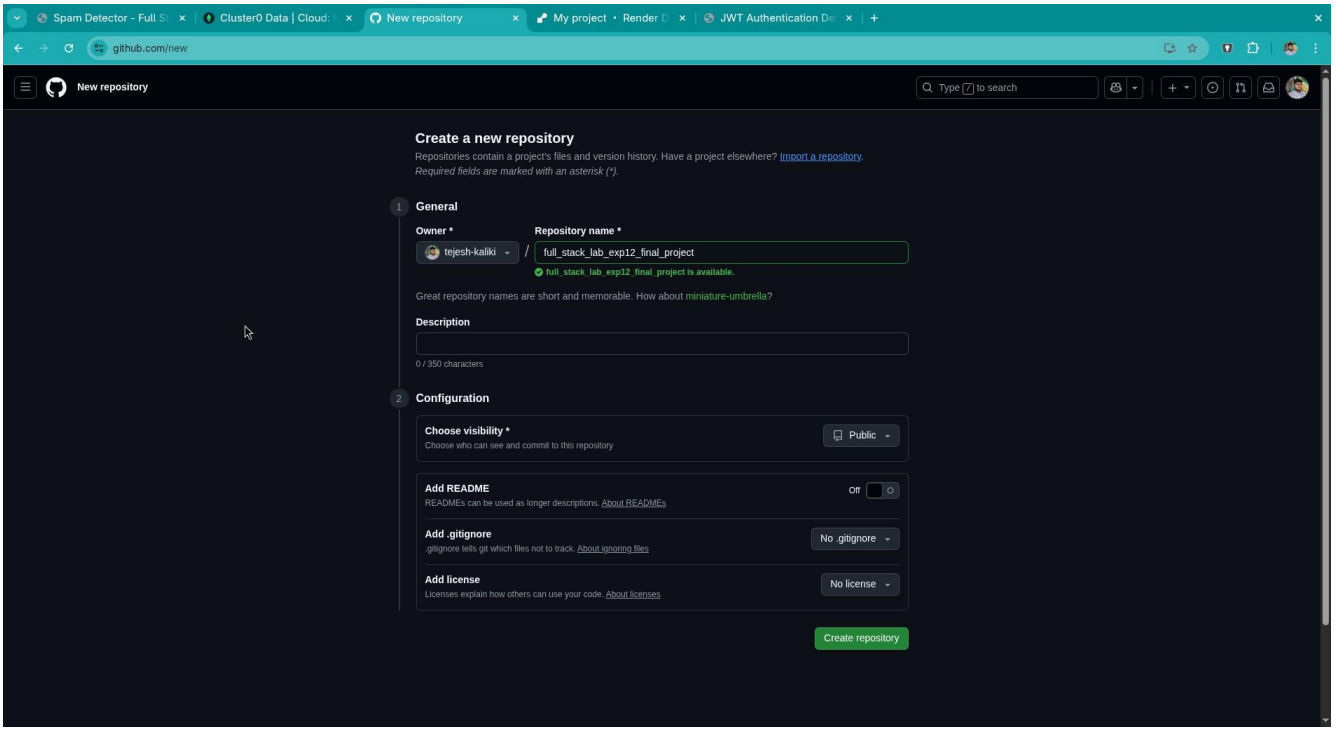
Output

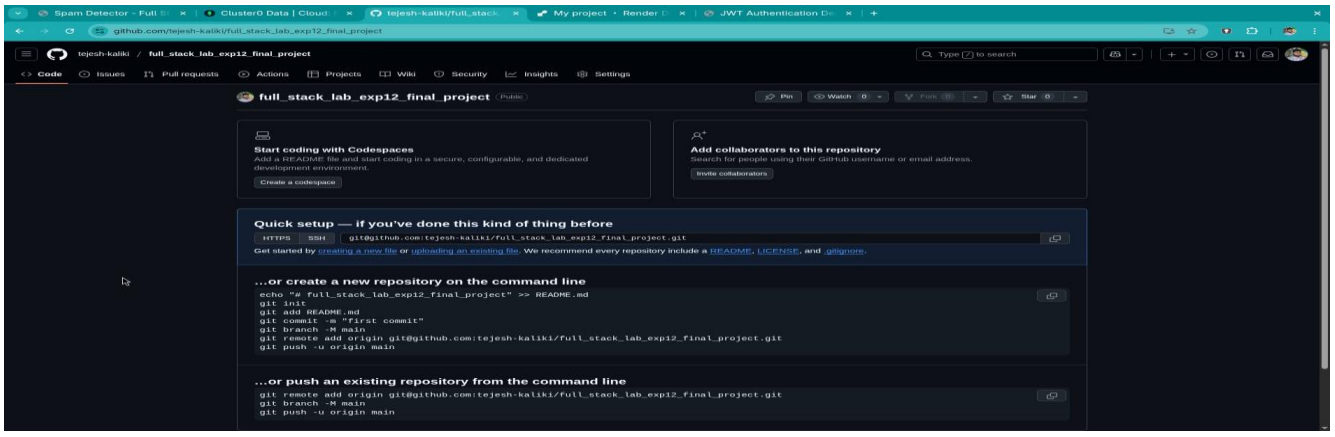
A production-ready deployed application with:

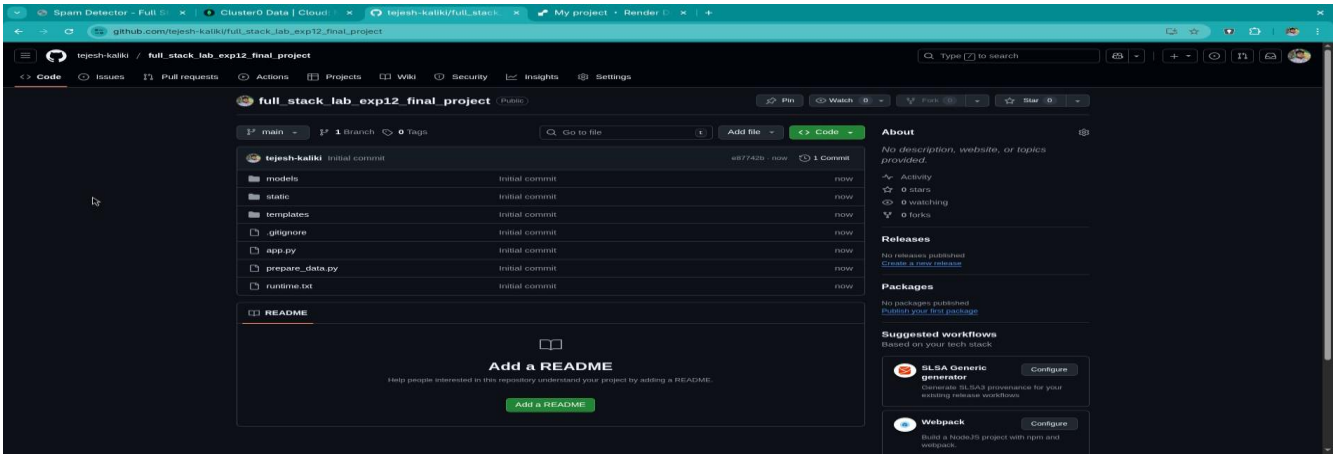
- Live public URL
- Automatic deployments on code push
- Environment-based configuration
- MongoDB Atlas integration
- HTTPS/SSL security
- Auto-scaling capabilities
- Application logs and monitoring
- Custom domain support (optional)
- Continuous deployment pipeline

Screen-shots

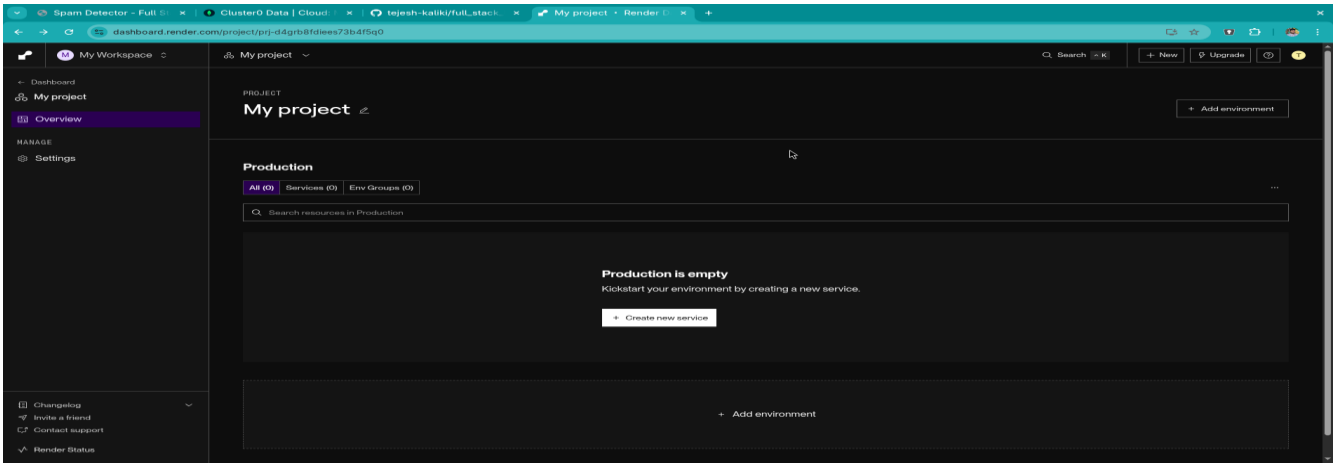
Setup project in github

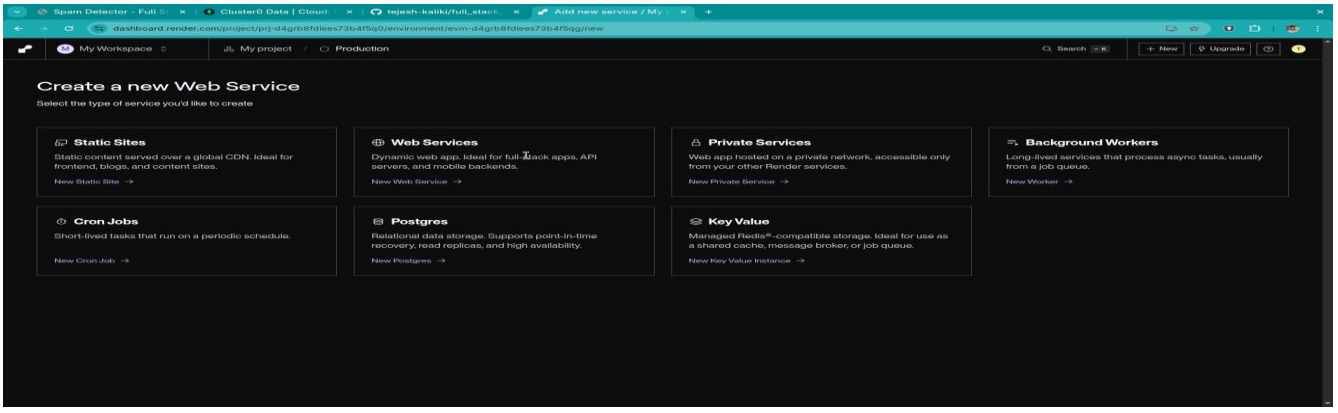


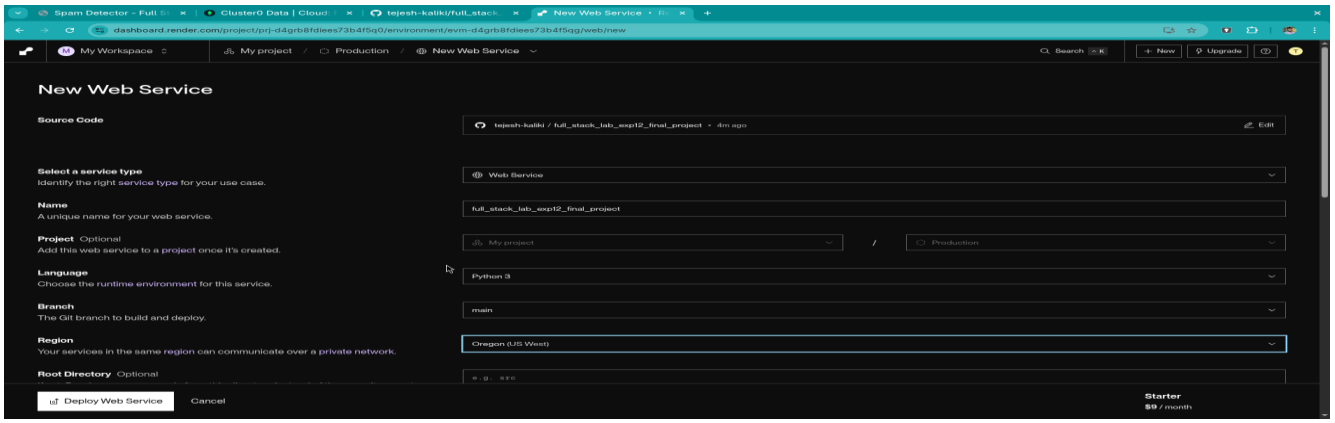


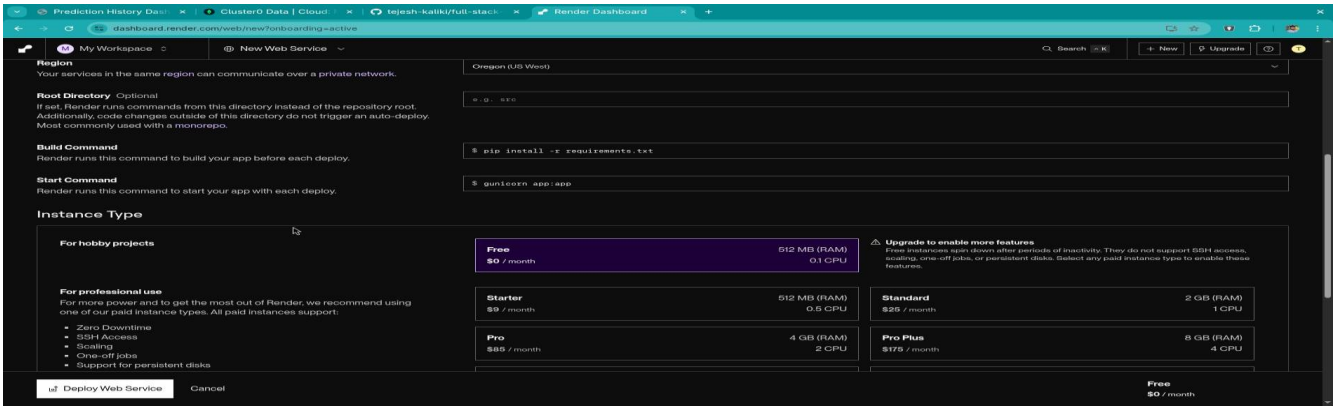


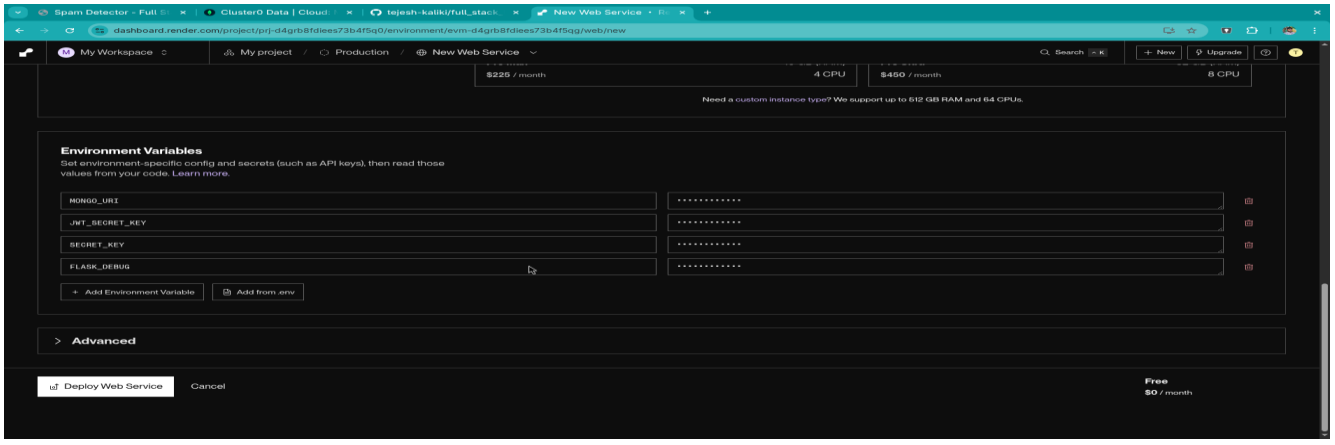
render.com setup

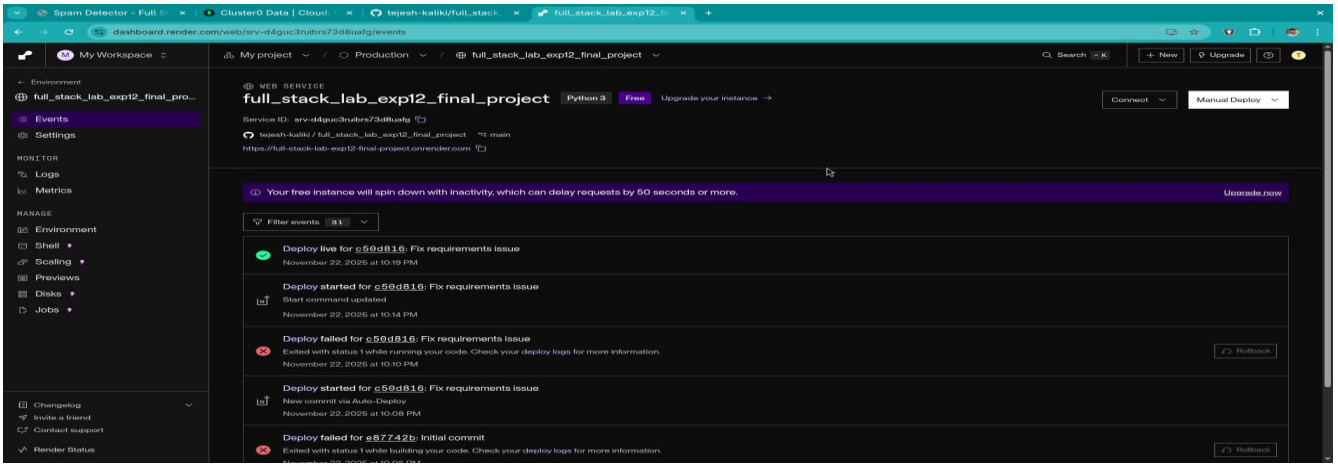


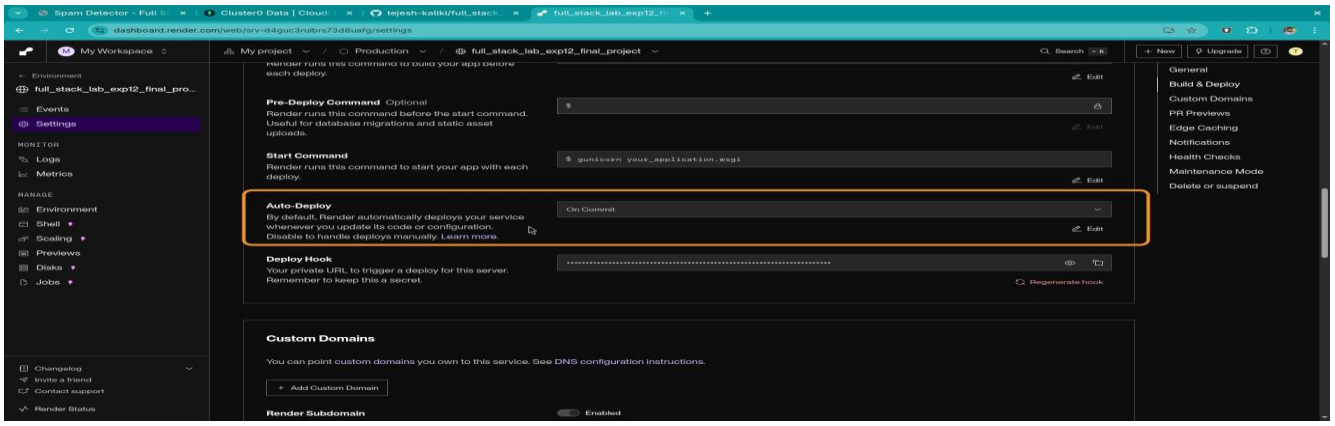



New Web Service
Source Code
tejesh-kaliki / full_stack_lab_exp12_final_project
Edit
Select a service type
Identify the right service type for your use case.
Web Service
Name
A unique name for your web service.
full_stack_lab_exp12_final_project
Project Optional
Add this web service to a project once it's created.
Language
Choose the runtime environment for this service.
Python 3
Branch
The Git branch to build and deploy.
main
Region
Your services in the same region can communicate over a private network.
Oregon (US West)
Root Directory Optional
Deploy Web Service
Cancel
Starter
$9 / month

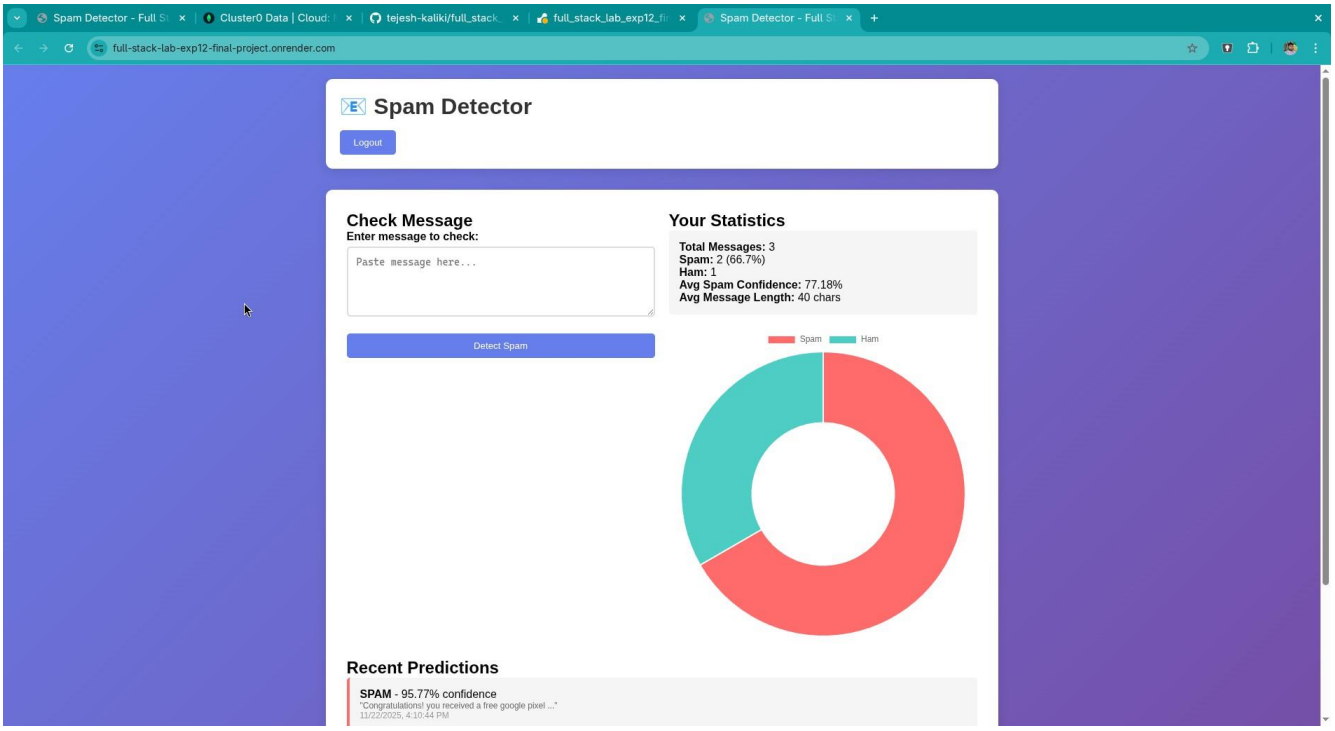

Region
Your services in the same region can communicate over a private network.
Oregon (US West)
Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a monorepo.
Build Command
Render runs this command to build your app before each deploy.
$ pip install -r requirements.txt
Start Command
Render runs this command to start your app with each deploy.
$ gunicorn app:app
Instance Type
For hobby projects
Free
$0 / month
512 MB (RAM)
0.1 CPU
Upgrade to enable more features
For professional use
For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:
Zero Downtime
SSH Access
Scaling
One-off jobs
Support for persistent disks
Starter
$9 / month
512 MB (RAM)
0.5 CPU
Standard
$25 / month
2 GB (RAM)
1 CPU
Pro
$85 / month
4 GB (RAM)
2 CPU
Pro Plus
$175 / month
8 GB (RAM)
4 CPU
Deploy Web Service
Cancel
Free
$0 / month


$225 / month
4 CPU
$450 / month
8 CPU
Need a custom instance type? We support up to 512 GB RAM and 64 CPUs.
Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. Learn more.
MONGO_URI
JWT_SECRET_KEY
SECRET_KEY
FLASK_DEBUG
Add Environment Variable
Add from .env
Advanced
Deploy Web Service
Cancel
Free
$0 / month


full_stack_lab_exp12_final_project
Python 3
Free
Upgrade your instance
Connect
Manual Deploy
Events
Settings
Logs
Metrics
Environment
Shell
Scaling
Previews
Disks
Jobs
Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.
Upgrade now
Deploy live for c50d816: Fix requirements issue
November 22, 2025 at 10:19 PM
Deploy started for c50d816: Fix requirements issue
Start command updated
November 22, 2025 at 10:14 PM
Deploy failed for c50d816: Fix requirements issue
Exited with status 1 while running your code. Check your deploy logs for more information.
November 22, 2025 at 10:10 PM
Deploy started for c50d816: Fix requirements issue
New commit via Auto-Deploy
November 22, 2025 at 10:08 PM
Deploy failed for e87742b: Initial commit
Exited with status 1 while building your code. Check your deploy logs for more information.



Final website verification



Result

Successfully deployed a complete full-stack AI application to production on Render, demonstrating cloud deployment and modern DevOps practices suitable for real-world applications.

Summary of All 12 Experiments

Exp	Topic	Key Concepts
1	Flask Setup	HTTP, routing, templates, static files
2	Authentication	MongoDB, bcrypt, sessions, user management
3	ML Training	scikit-learn, Iris dataset, model serialization
4	ML API	REST APIs, JSON responses, Flask blueprints
5	Frontend Integration	AJAX, fetch API, real-time predictions
6	Dashboard	Chart.js, data visualization, MongoDB aggregation
7	JWT Auth	Stateless authentication, token refresh, token revocation
8	File Upload	File handling, image processing, Pillow
9	History & Viz	Data persistence, aggregation pipelines, trends
10	CI/CD	GitHub Actions, automated testing, Render deployment
11	Mini-Project	End-to-end ML app, spam detection, TF-IDF
12	Deployment	Production deployment, environment config, monitoring

---

Prerequisites Summary

Python Libraries

```
flask   pymongo
scikit-learn bcrypt
python-dotenv
flask-jwt-extended pillow
gunicorn
pandas
numpy
```

Services

- MongoDB Atlas (Cloud database)
- Render.com (Web hosting)
- GitHub (Code repository)

Skills Covered

- Backend development with Flask
- Database design and queries
- Machine learning model training and deployment
- Frontend development with HTML/CSS/JS
- RESTful API design
- Authentication and authorization
- Cloud deployment
- CI/CD pipeline setup
- Real-world application development